

```

library(dplyr)
library(randomForest)
library(tibble)
library(ranger)
library(glmnet)
library(catboost)
library(rsample)
library(purrr)

list_se <- readRDS("list_se.rds")
final_matrices <- readRDS("final_matrices.rds")
changes <- readRDS("changes_data_cleaning.rds")

wang_signature_genes <- c("Ccng1", "Dgka", "Fzr1", "H4c3", "H4c9", "Ifit1", "Igfbp4",
"LOC118567921",
                        "Lrrc70", "Ms4a1", "Phlda3", "Ptpn", "Rps20", "Serpine2", "Thy1")

set.seed(42)

# Read Wang Training Data
wang_training_data <- read.csv("wang_training_dataset.csv", header = FALSE)
wang_training_data <- as.data.frame(t(wang_training_data))
colnames(wang_training_data) <- wang_training_data[1,]
wang_training_data <- wang_training_data[-1,]
wang_training_data <- type.convert(wang_training_data, as.is = TRUE)

# Read Wang Test Data
wang_testing_data <- read.csv("wang_testing_dataset.csv", header = FALSE)
wang_testing_data <- as.data.frame(t(wang_testing_data))
colnames(wang_testing_data) <- wang_testing_data[1,]
wang_testing_data <- wang_testing_data[-1,]
wang_testing_data <- type.convert(wang_testing_data, as.is = TRUE)

# Gene Selection
wang_training_data <- wang_training_data %>% dplyr::select(c("Dose",
all_of(wang_signature_genes)))
wang_testing_data <- wang_testing_data %>% dplyr::select(c("Dose",
all_of(wang_signature_genes)))

# Linear Regression Model
linear_model <- lm(Dose ~ ., data = wang_training_data)

# Random Forest Regression (version with ranger)
random_forest_model <- ranger::ranger(
  Dose ~ .,
  data = wang_training_data,
  num.trees = 60,
  seed = 42,
  max.depth = 2,
  splitrule = "variance"
)

# Polynomial Linear Model
wang_poly <- wang_training_data %>%
  dplyr::mutate(across(2:last_col(), ~ .x^2))
polynomial_model <- lm(Dose ~ ., data = wang_poly)

# Square Root Linear Model
wang_root <- wang_training_data %>%
  dplyr::mutate(across(2:last_col(), ~ sqrt(.x)))
square_root_model <- lm(Dose ~ ., data = wang_root)

# ElasticNet Regression Model
elastic_model <- glmnet::glmnet(
  x = as.matrix(wang_training_data[, 2:ncol(wang_training_data)]),
  y = wang_training_data$Dose,
  alpha = 0.5,
  lambda = 1.0,
  standardize = FALSE,
  thresh = 0.0002
)

```

```

)

# Data augmentation

wang_training_data_augmented <- wang_training_data %>%
  mutate(Target = case_when(
    Dose == 0 ~ 0,
    Dose < 4 ~ 2,
    Dose < 6 ~ 4,
    Dose < 9 ~ 6,
    Dose < 12 ~ 9,
    TRUE ~ 12,
  )) %>%
  relocate(Target, 1) %>%
  dplyr::group_by(Target) %>%
  dplyr::mutate(across(-Dose, ~ .x + runif(n = 1, min = -0.1, max = 0.1)*sd(.x))) %>%
  dplyr::ungroup()

wang_training_data_augmented$Target <- NULL

augmented_set <- rbind(wang_training_data_augmented, wang_training_data)
rownames(augmented_set) <- NULL
augmented_set$Sample_id <- seq_len(nrow(augmented_set))
augmented_set <- augmented_set %>% relocate(Sample_id, 1)

# Cross Validation

cross_validation <- rsample::vfold_cv(augmented_set, v = 5, repeats = 10)

predictions <- lapply(seq_along(cross_validation$splits), function(i) {
  splits <- cross_validation$splits[[i]]
  repetition <- cross_validation$id[i]
  fold <- cross_validation$id2[i]

  testing_folds <- rsample::assessment(splits)
  training_folds <- rsample::training(splits)

  # Simple Linear model
  linear_model <- lm(Dose ~ . - Sample_id, data = training_folds)

  # Polynomial Linear Model
  wang_poly <- training_folds %>%
    dplyr::mutate(across(3:last_col(), ~ .x^2))
  testing_poly <- testing_folds %>%
    dplyr::mutate(across(3:last_col(), ~ .x^2))
  polynomial_model <- lm(Dose ~ . - Sample_id, data = wang_poly)

  # Square Root Linear Model
  wang_root <- training_folds %>%
    dplyr::mutate(across(3:last_col(), ~ sqrt(.x)))
  testing_root <- testing_folds %>%
    dplyr::mutate(across(3:last_col(), ~ sqrt(.x)))
  square_root_model <- lm(Dose ~ . - Sample_id, data = wang_root)

  # Random Forest Regression
  random_forest_model <- ranger::ranger(
    Dose ~ . - Sample_id,
    data = training_folds,
    num.trees = 60,
    seed = 42,
    max.depth = 2,
    splitrule = "variance"
  )

  # ElasticNet Regression Model
  elastic_model <- glmnet::glmnet(
    x = as.matrix(training_folds[, 3:ncol(training_folds)]),
    y = training_folds$Dose,

```

```

alpha = 0.5,
lambda = 1.0,
standardize = FALSE,
thresh = 0.0002
)

catboost_matrix <- data.frame(
  Sample_id = training_folds$Sample_id,
  Dose = training_folds$Dose,
  Linear = predict(linear_model, newdata = training_folds),
  Polynomial = predict(polynomial_model, newdata = wang_poly),
  SquareRoot = predict(square_root_model, newdata = wang_root),
  RandomForest = predict(random_forest_model,
                        data = training_folds)$predictions,
  ElasticNet =
    as.numeric(glmnet::predict.glmnet(elastic_model,
                                     newx = as.matrix(training_folds[,
3:ncol(training_folds)])))
)

#Meta-model: CatBoost Regression Model
catboost_model <- catboost::catboost.train(
  learn_pool = catboost::catboost.load_pool(
    data = as.matrix(catboost_matrix[, 3:7]),
    label = catboost_matrix$Dose
  ),
  params = list(
    loss_function = "RMSE",
    depth = 2,
    iterations = 50,
    learning_rate = 0.08,
    l2_leaf_reg = 4.5,
    subsample = 0.75,
    random_seed = 42
  )
)

# Predict testing_folds with the base trained on training_folds
testing_matrix <- data.frame(
  Sample_id = testing_folds$Sample_id,
  Dose = testing_folds$Dose,
  Linear = predict(linear_model, newdata = testing_folds),
  Polynomial = predict(polynomial_model, newdata = testing_poly),
  SquareRoot = predict(square_root_model, newdata = testing_root),
  RandomForest = predict(random_forest_model,
                        data = testing_folds)$predictions,
  ElasticNet =
    as.numeric(glmnet::predict.glmnet(elastic_model,
                                     newx = as.matrix(testing_folds[,
3:ncol(testing_folds)])))
)

# Predict testing_folds with the catboost trained on training_folds
testing_catboost <- catboost.predict(
  catboost_model,
  catboost.load_pool(
    data = as.matrix(testing_matrix[, 3:7])
  )
)

# Performance metrics
real_dose = testing_folds$Dose
predicted_dose = testing_catboost

predictions <- data.frame(
  Repetition = repetition,
  Fold = fold,
  Sample_id = testing_folds$Sample_id,
  Dose = real_dose,
  Linear = predict(linear_model, newdata = testing_folds),

```

```

Polynomial = predict(polynomial_model, newdata = testing_poly),
SquareRoot = predict(square_root_model, newdata = testing_root),
RandomForest = predict(random_forest_model,
                        data = testing_folds)$predictions,
ElasticNet =
  as.numeric(glmnet::predict.glmnet(elastic_model,
                                   newx = as.matrix(testing_folds[,
3:ncol(testing_folds)]))),
TempCatBoost = predicted_dose,
RMSE = sqrt(mean((real_dose - predicted_dose)^2)),
MAE = mean(abs(real_dose - predicted_dose)),
AbsError = abs(real_dose - predicted_dose),
RSquared = 1 - sum((real_dose - predicted_dose)^2) /
  sum((real_dose - mean(real_dose))^2),
RelativeError = ifelse(
  real_dose == 0,
  NA,
  abs((predicted_dose - real_dose) / real_dose) * 100
))
return(predictions)
})

#####
#####

final_predictions_matrix <- predictions %>%
  purrr::list_rbind() %>%
  dplyr::select(Sample_id, Dose, Linear, Polynomial, SquareRoot, RandomForest, ElasticNet) %>%
  dplyr::arrange(Sample_id) %>%
  tibble::remove_rownames() %>%
  dplyr::group_by(Sample_id) %>%
  dplyr::summarise(across(everything(), mean)) %>%
  dplyr::ungroup()

performance_matrix <- predictions %>%
  purrr::list_rbind() %>%
  dplyr::group_by(Repetition, Fold) %>%
  dplyr::summarise(
    RMSE = first(RMSE),
    MAE = first(MAE),
    RSquared = first(RSquared),
    MRE = mean(RelativeError, na.rm = TRUE),
    REmax = max(RelativeError, na.rm = TRUE),
    .groups = "drop"
  )

# Confidence Intervals
confidence_interval <- function(performance_matrix, x) {
  mean <- mean(performance_matrix[[x]])
  n <- length(performance_matrix[[x]])
  sd <- sd(performance_matrix[[x]])
  se <- sd / sqrt(n)

  alpha = 0.05
  degrees.freedom = n - 1
  t.score = qt(p = alpha / 2, df=degrees.freedom, lower.tail = FALSE)

  margin.error <- t.score * se

  lower_bound <- mean - margin.error
  upper_bound <- mean + margin.error
  print(c(lower_bound, upper_bound))
}

CI_RMSE <- confidence_interval(performance_matrix, x = "RMSE")
CI_MAE <- confidence_interval(performance_matrix, x = "MAE")
CI_RSquared <- confidence_interval(performance_matrix, x = "RSquared")
CI_MRE <- confidence_interval(performance_matrix, x = "MRE")

```

```

CI_REmax <- confidence_interval(performance_matrix, x = "REmax")

# Meta-model: CatBoost Regression
# Trained on the averaged matrix
final_catboost_model <- catboost::catboost.train(
  learn_pool = catboost::catboost.load_pool(
    data = as.matrix(final_predictions_matrix[, 3:7]),
    label = final_predictions_matrix$Dose
  ),
  params = list(
    loss_function = "RMSE",
    depth = 2,
    iterations = 50,
    learning_rate = 0.08,
    l2_leaf_reg = 4.5,
    subsample = 0.75
  )
)

# Linear model transformations
testing_poly <- wang_testing_data %>%
  dplyr::mutate(across(2:last_col(), ~ .x^2))
testing_root <- wang_testing_data %>%
  dplyr::mutate(across(2:last_col(), ~ sqrt(.x)))

# Predicting the testing data
testing_matrix <- data.frame(
  Dose = wang_testing_data$Dose,
  Linear = predict(linear_model, newdata = wang_testing_data),
  Polynomial = predict(polynomial_model, newdata = testing_poly),
  SquareRoot = predict(square_root_model, newdata = testing_root),
  RandomForest = predict(random_forest_model,
    data = wang_testing_data)$predictions,
  ElasticNet =
    as.numeric(glmnet::predict.glmnet(elastic_model,
      newx = as.matrix(wang_testing_data[,
        2:ncol(wang_testing_data)])))
)

# Predict with catboost
catboost_predictions <- catboost.predict(
  final_catboost_model,
  catboost.load_pool(
    data = as.matrix(testing_matrix[, 2:6])
  )
)
testing_matrix$CatBoost <- catboost_predictions

# Final performance metrics
rownames(testing_matrix) <- NULL
testing_matrix$RMSE <- sqrt(mean((testing_matrix$Dose - testing_matrix$CatBoost)^2))
testing_matrix$MAE <- mean(abs(testing_matrix$Dose - testing_matrix$CatBoost))
testing_matrix$AbsError <- abs(testing_matrix$Dose - testing_matrix$CatBoost)
testing_matrix$RSquared <- 1 - sum((testing_matrix$Dose - testing_matrix$CatBoost)^2) /
  sum((testing_matrix$Dose - mean(testing_matrix$Dose))^2)
testing_matrix$RelativeError <- ifelse(testing_matrix$Dose == 0, NA,
  abs((testing_matrix$CatBoost - testing_matrix$Dose) /
testing_matrix$Dose) * 100)

my_RMSE <- dplyr::first(testing_matrix$RMSE)
my_MAE <- dplyr::first(testing_matrix$MAE)
my_RSquared <- dplyr::first(testing_matrix$RSquared)
my_MRE <- mean(testing_matrix$RelativeError, na.rm = TRUE)
my_REmax <- max(testing_matrix$RelativeError, na.rm = TRUE)

wang_RMSE <- 0.97
wang_MAE <- 0.785
wang_RSquared <- 0.949
wang_MRE <- 11.138

```

```
wang_REmax <- 28.687
```

```
# Comparison Table
```

```
comparison_table <- data.frame(  
  Metric = c("RMSE", "MAE", "R-Squared", "MRE", "REmax"),  
  My_Model = c(my_RMSE, my_MAE, my_RSquared, my_MRE, my_REmax),  
  Original_Model = c(wang_RMSE, wang_MAE, wang_RSquared, wang_MRE, wang_REmax))  
print(comparison_table, row.names = FALSE)
```